CSE 143 Final Project Report

Spring 2026, team_01

Raj Nadkarni	**Sebastian Woo**	**Leo Li**	**Ashton Liu**
ranadkar	seewoo	bli312	ascliu

Overview

We implemented and evaluated a simple statistical watermark for LLM generated text. We used the "green list" watermarking idea from Kirchenbauer et al. [1]. During generation, for each token, we derive a pseudorandom subset of the vocabulary, which we call the "green list." Tokens in the green list are called "green tokens," and they get a logit boost before sampling. This makes watermarked text indistinguishable to human readers, since it still looks like normal model text. But statistically, it will contain more green tokens than normal unwatermarked text. Premature Abstraction's video [2] provides a more detailed explanation of the scheme.

We also built a detector which given a piece of text, tokenizes it, recomputes the expected green list for each previous token, counts how often the next token falls in the greenlist, and reports a z-score under the baseline assumption that tokens are green only with probability γ. If the z-score is above our threshold, we classify the text as watermarked.

There are many reasons why being able to determine whether or not outputs come from a particular LLM is useful. Outside the obvious catching cheaters in academics, there are some implications with upholding copyright and compliance laws. Another use case is in cleaning training data. Nowadays, pretty much all frontier LLMs are trained on the entire open internet. But a significant proportion of the internet was generated by LLMs. Training on previously generated content can lead to model collapse, and watermarking can be an effective tool in filtering out such data when cleaning. Provenance detection is also the motivation behind earlier neural text watermarking work such as Abdelnabi and Fritz [3]. We are also interested in the limitations of this idea, especially in regards to whether simple text transformations can remove the statistical signal.

Implementation

Our implementation is in `src/watermark/greenlist.py`, with a CLI driver in `main.py`.

We used the following main parameters:

parameter	**value**	**meaning**
`gamma`	`0.25`	Proportion of the vocabulary placed in the greenlist for each context
`delta`	`3.0`	Logit boost for green tokens
`secret_key`	`15485863`	Private key used to derive deterministic green lists (this specific value is the 1 millionth prime number)
`z_threshold`	`4.0`	Minimum detector z-score to call text watermarked

To derive the green list, we hash the secret key and the token id, use it to seed a torch generator, then sample a random permutation of the vocabulary and use the first $\gamma\ |V|$ tokens as the green list. So, the green list selection is pseudorandom and deterministic.

For generation, we use the `transformers` library from HuggingFace. We pass a `LogitsProcessor` into `model.generate` which modifies the score tensor before sampling. For each batch item, it looks at the

most recent token, derives (or retrieves from cache) the green list, and adds `delta` to the logits of its green tokens.

The default model we use is `Qwen/Qwen3-30B-A3B-Instruct-2507` from HuggingFace. We also tested and confirmed it works with `Qwen3-32B` and `Qwen2.5-72B-Instruct`.

The detector scores a sequence of token ids

$$Y_i = \begin{cases} 1 & \text{if token } \ i \ \text{ is green} \\ 0 & \text{otherwise} \end{cases}$$

Under the assumption that the text was not generated with our watermark, subsequent tokens should land in their green lists with probability γ, so we model each Y_i as a Bernoulli random variable with mean γ. Define the total green token count as

$$X = \sum_{i=1}^{T} Y_i$$

Observe that since X is the sum of T Bernoulli trials, we can model it as a binomial random variable

$$X \sim \text{Binomial}(T, \gamma)$$

Which gives

$$E[X] = T\gamma$$
$$\text{Var}[X] = T\gamma(1-\gamma)$$

Our code directly follows this calculation in `GreenListWatermark.score_token_ids`:

```
expected = total * self.config.gamma
variance = total * self.config.gamma * (1.0 - self.config.gamma)
z_score = (green_count - expected) / math.sqrt(variance)
p_value = 0.5 * math.erfc(z_score / math.sqrt(2.0))
```

We also report the green fraction, and a one sided p-value. And, we have logic to skip invalid token ids as well as negative results for inputs that are too short to score (like less than two tokens).

The final decision is made by comparing this z-score to our configured threshold:

```
is_watermarked=z_score >= self.config.z_threshold
```

So for our default configuration, the detector only returns true when the observed green-token count is at least four standard deviations above what we would expect from ordinary, unwatermarked text.

Experiments And Findings

Here is an example run, with the number of output tokens capped at `500`. The prompt was:

```
uv run main.py generate --prompt "tell me when I would use a b+ tree vs an lsm tree"
```

Unwatermarked response:

```
Great question! B+ trees and LSM (Log-Structured Merge) trees are both data
structures used to efficiently manage large, ordered datasets in databases and file
systems. However, they have different strengths and trade-offs. Here's a breakdown of
when to use each:

Use a B+ Tree When:
- High read-heavy workloads with frequent range queries
```

```
- Low to moderate write workloads
- Random access patterns with low latency requirements
```

Watermarked response:

```
Choosing between a B+ tree and an LSM (Log-Structured Merge) tree depends heavily on
your workload, performance goals, and storage requirements. Here's a breakdown of
when to use each:

Use a B+ Tree when:
- High read performance is critical
- Workloads are read-heavy
- Low write amplification and minimal space overhead
```

Both outputs are quite natural and correctly answered the prompt. But the detector doesn't care about such things, and the numbers don't lie. (Keep in mind the expected green fraction for naturally occuring text is approximately `0.250`):

output	scored tokens	green count	green fraction	z-score	p-value	verdict
Unwatermarked	499	128	0.257	0.34	0.368	False
Watermarked	499	173	0.347	4.99	3.05e-07	True

The example shows the behavior we expected. The unwatermarked sample had a green fraction very close to the null hypothesis of $\gamma = 0.25$ and a low z-score. The watermarked sample had the same number of scored tokens, but 45 more green tokens than the unwatermarked sample, pushing the z-score above the threshold of 4.0.

We also tested some attacks on the watermark. We didn't make a full benchmark; instead we just evaluated it qualitatively, but our results clearly showed that most transformations that preserve the meaning while changing the surface form are very effective in scrubbing the watermark.

trial	procedure	detector result
Unwatermarked generation	Generated normally with no logits processor	False
Watermarked generation	Generated with green token logit boosting	True
Translation roundtrip	Translated watermarked text into another language and back into English.	False
LLM paraphrasing	Asked another LLM to rewrite watermarked text	False
Small manual edits	Edited or replaced individual words by hand	True

The main takeaway here is that this watermark is only really useful when the generated text is mostly preserved. Our detector depends on exact token level stats, so any rewrites that change most bigrams will remove the signal. There are other watermarking schemes that attempt to address these issues such as Kuditipudi et al. [4], but we did not get to explore them in depth. There is obviously also a tradeoff between detectability and the quality of generated text. Increasing `delta` makes the detector stronger since the model chooses green tokens more often, but this also interferes with the model's distribution. A lower `delta` preserves the underlying distribution much more but makes detection harder, particularly for shorter texts.

Lastly, we tried a watermark stealing approach similar to the one from Jovanovic et al. [5]. They were able to demonstrate that it is possible to steal a watermark using under $50 in ChatGPT API credits. However, we were not able to achieve the same results on our self-hosted model.

Code Description And How To Run

Code files

- `src/watermark/greenlist.py`: Main watermark implementation. Defines the watermark configuration, deterministic green list generation, logits processor, model loading, text generation, and detection scoring.
- `src/watermark/__init__.py`: Export the public watermark API.
- `main.py`: CLI driver for generating watermarked text and detecting watermarks in text or files.
- `tests/test_greenlist.py`: Unit tests for miscellaneous stuff.

How to Run

Make sure uv is installed and in the path.

Under the `project/` directory:

```
# install dependencies
uv sync --locked

# run tests
uv run pytest

# generate completions
uv run main.py generate --prompt "tell me about ..."

# detect from direct text
uv run main.py detect --text "text to score"

# detect from text file
uv run main.py detect --file content.txt
```

Note that we ran our experiments on a server with 8 NVIDIA H200 GPUs, totalling approximately 1.1 TiB VRAM. Make sure your machine has enough GPU memory for the selected HuggingFace model before running generation.

Task Distribution

We worked together in a VS Code Live Share session while on a Discord call or in person. We jointly discussed the paper ideas, implemented the green list algorithm, tested detector behavior, and tried attacks such as translation round trip and LLM paraphrasing.

Bibliography

[1] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein, "A Watermark for Large Language Models," Jan. 24, 2023, *arXiv*. doi: 10.48550/arXiv.2301.10226.

[2] Premature Abstraction, "How to Watermark Text (LLM Watermarking Explained)." [Online]. Available: https://youtu.be/55LXhbPjyeM

[3] Sahar Abdelnabi and Mario Fritz, "Adversarial Watermarking Transformer: Towards Tracing Text Provenance with Data Hiding," Sept. 07, 2020, *arXiv*. doi: 10.48550/arXiv.2009.03015.

[4] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang, “Robust Distortion-free Watermarks for Language Models,” July 28, 2023, *arXiv*. doi: 10.48550/arXiv.2307.15593.

[5] Nikola Jovanovic, Robin Staab, and Martin Vechev, “Watermark Stealing in Large Language Models,” Feb. 29, 2024, *arXiv*. doi: 10.48550/arXiv.2402.19361.